ECAP-RAG

Product Requirements Document

Evidence-Grounded, Contradiction-Aware PathRAG

DOCUMENT VERSION	2.1
UPDATE SCOPE	Module 2 implementation and validation record
PROJECT STATUS	Week 2 complete; Week 3 traversal work next
TARGET ROADMAP	8-week engineering plan
REVISION DATE	24 September 2026

MODULE 2 OUTCOME

A connected, provenance-aware SVO MultiDiGraph now stores validated factual claims on both edges and participating entity nodes, with exact document, chunk, sentence, and character-span evidence.

Document Control

Version	Change	Status
2.0	Transition from attention slicing to provenance and contradiction graph architecture.	Superseded
2.1	Module 2 updated to match the implemented graph builder, outputs, tests, and one-command execution workflow.	Current

Contents

1	Executive Summary and Core Intent
2	Technical Flaws and Proposed Solutions
3	System Architecture and End-to-End Pipeline
4	Technical Specifications and Module Requirements
4.2	Module 2 Implementation Record
5	Mathematical Formulations Reference
6	8-Week Engineering Implementation Roadmap
7	Success and Acceptance Criteria

Update Summary

- Module 2 status changed from **Current Focus** to **Implemented and validated**.
- The design now specifies a **NetworkX MultiDiGraph**, allowing distinct factual claims between the same entity pair.
- Entity nodes retain aliases, mentions, claim IDs, full participating claim records, and supporting chunk IDs.
- A sparse, connected traversal backbone is generated separately from the complete evidence graph.
- The implemented sample passed **9 of 9** structural, provenance, alias, and retrieval checks.
- A single `main.py` command builds, exports, evaluates, and plots the graph. This is a Module 2 utility and does not complete the Week 7 LLM orchestrator.

1. Executive Summary and Core Intent

Standard RAG architectures often provide weak multi-hop reasoning, incomplete factual provenance, and no principled handling of conflicting claims. ECAP-RAG replaces opaque context compression and independent vector chunks with a provenance-aware knowledge graph, contradiction-aware path retrieval, and text-level context pruning.

Architectural Guarantees

- **Provenance and traceability:** Every accepted factual claim maps to a document ID, chunk ID, sentence ID, and exact source span.
- **Contradiction awareness:** Later modules will preserve conflicting claim-level evidence and present it as structured supporting and opposing context.
- **Text-level deduplication and pruning:** Later modules will remove redundant chunks and irrelevant sentences before the final LLM request.
- **Traversal-ready graph:** Module 2 now provides both the full evidence graph and a sparse connected backbone suitable for bounded path retrieval.

2. Technical Flaws and Proposed Solutions

Flaw	Baseline limitation	ECAP-RAG response
Unanchored graph claims	Claims are separated from their source text.	Span-level provenance on every accepted claim and claim edge.
Multi-hop reasoning failures	Chunks are evaluated independently, losing bridge evidence.	Bounded traversal over a connected entity-anchored graph.
Contradiction blindsiding	Conflicts are treated as noise or discarded.	Claim-level contradiction detection and qualified evidence retention.
Redundant context payloads	Repeated chunks inflate the prompt.	CPU deduplication and sentence-level pruning.
Query anchor mismatch	A single query string misses entities and relations.	Two-tier entity and relation anchors from Module 1.

3. System Architecture and End-to-End Pipeline

1	User query
2	Tier-1 entity anchors and Tier-2 relation anchors
3	Seed mapping to the provenance-aware knowledge graph
4	Bounded multi-anchor path retrieval
5	Path scoring and contradiction analysis
6	Traceback to exact chunks and spans
7	Deduplication and sentence-level context pruning
8	Final LLM generation with citations

Module 2 supplies step 3's graph foundation. Module 3 remains responsible for query-time bounded path search; Module 2's traversal backbone is an indexing and visualization artifact, not a replacement for query-aware path generation.

4. Technical Specifications and Module Requirements

4.1 Module 1: Two-Tier Query Parser

File: `src/query_parser.py` **Status:** Complete (Week 1)

Converts free-form queries into a Multi-Anchor Query Graph containing disjoint entity and relation anchor sets.

- Sanitize query text and remove non-informative noise.
- Extract Tier-1 named-entity anchors using spaCy NER.
- Extract Tier-2 verb and relation anchors from the dependency parse.
- Return a structured object such as `{"entities": [...], "relations": [...]}`.

4.2 MODULE 2	IMPLEMENTED AND VALIDATED
Provenance-Aware Knowledge Graph Builder	**Week 2 complete**

Implemented files: `ecap_kg/graph_builder.py`, `ecap_kg/evaluate_graph.py`, `ecap_kg/retrieve_graph.py`, `ecap_kg/plot_graph.py`, and `ecap_kg/main.py`.

Purpose: Ingest a UTF-8 text document and build a connected, provenance-grounded NetworkX `MultiDiGraph` whose entity nodes and factual edges both retain exact source evidence.

4.2.1 Implemented Processing Pipeline

- **Document cleaning and stable identity:** Removes supported markup noise, assigns a deterministic document ID, and preserves normalized source text for provenance validation.
- **In-memory chunking:** Creates bounded chunks with stable chunk IDs and exact document offsets. Chunks are serialized with the graph; SQLite is intentionally not required for Module 2.
- **Entity and canonical resolution:** Normalizes aliases such as 'quantum mechanics' and 'quantum physics' into one canonical entity while retaining the original aliases and mentions.
- **Conservative reference resolution:** Resolves supported pronouns and deictic references from recent entity context. Unresolved references are rejected rather than becoming nodes such as 'this', 'which', or 'another'.
- **Noun-phrase boundary control:** Produces compact subject and object phrases and handles supported relative and subordinate clause patterns without promoting malformed fragments.
- **Atomic SVO extraction:** Converts dependency structures into subject-relation-object claims with confidence, voice, negation, extraction method, and evidence spans.
- **Triple validation:** Rejects unresolved references, invalid phrases, self-referential triples, and candidates below the confidence threshold. Rejections remain inspectable in the export.
- **Graph construction:** Stores each accepted claim as a distinct factual edge and copies the complete participating claim record onto both entity nodes.
- **Connectivity repair:** Connects otherwise separate factual components with explicitly marked document-context edges. These edges support navigation but are never returned as source facts.
- **Traversal backbone:** Derives a sparse, maximum-confidence connected backbone with exactly N-1 edges for N nodes, while preserving the complete evidence graph separately.

4.2.2 Graph and Provenance Data Contract

Object	Required content	Role
Entity node	canonical_name, ent_type, aliases, mentions, claim_ids, claims, supporting_chunk_ids	Entity anchor plus direct factual context
Factual edge	relation, claim_id, confidence, negated, document_id, chunk_id, sentence_id, span, supporting_text	One validated SVO assertion
Claim record	subject/object IDs and labels, predicate, fact text, confidence, voice, spans, evidence text	Canonical provenance-bearing fact
Context bridge	inferred=true, edge_type=document_context, explicit context relation	Connectivity only; never factual evidence
Chunk	chunk_id, document_id, text, start/end offsets	Local evidence container

4.2.3 Serialized Outputs

Artifact	Contents
data/graph.pkl	Complete Python state including full MultiDiGraph, traversal_graph, chunks, claims, context bridges, rejected triples, aliases, and diagnostics.
data/graph.json	Readable JSON export containing node, edge, traversal, chunk, claim, rejection, and diagnostic records.
data/evaluation.json	Machine-readable evaluation report created by the unified runner.
data/knowledge_graph.png	Traversal visualization. Shell layout is selected with `--layout shell`; hierarchical, spring, and Kamada-Kawai modes remain available.

4.2.4 Unified Execution

```
.\.venv\Scripts\python.exe .\ecap_kg\main.py --layout shell --show --strict
```

The command reads the source document, builds the graph, writes pickle and JSON outputs, runs the evaluation suite, and saves the shell-layout traversal visualization. The current retrieval cases are document-specific; unrelated documents can use `--skip-evaluation` until their own expected queries are defined.

4.2.5 Validation Evidence

Metric	Observed result
Full graph	49 nodes, 41 factual claims, 7 marked context bridges, 1 weakly connected component
Traversal backbone	49 nodes, 48 edges, 1 component
Automated checks	9 passed of 9 executed

Passed checks

- Single connected component
- Sparse connected traversal backbone
- No unresolved generic or deictic nodes
- Quantum physics and quantum mechanics alias merge
- Claim provenance round trip
- Heisenberg formulated the uncertainty principle retrieval
- Einstein explained the photoelectric effect retrieval

- Quantum mechanics underpins technologies retrieval
- Quantum computing leverages superposition retrieval

CURRENT BOUNDARIES

Coreference resolution is deliberately conservative and heuristic rather than a general neural coreference system. The included retrieval suite is specific to the quantum-physics fixture. Query-time bounded multi-anchor traversal, path scoring, contradiction analysis, source traceback, pruning, and final LLM generation remain assigned to Modules 3 through 7.

4.3 Module 3: Bounded Path Traverser

File: `src/graph_traverser.py` **Status:** In scope - Week 3

- Map Tier-1 entities and Tier-2 relations to graph seed nodes.
- Run bounded multi-source BFS or k-shortest path search up to three hops.
- Build candidate paths that retain bridge entities, claim IDs, weights, and provenance pointers.
- Use the Module 2 traversal backbone as an optional navigation aid while consulting the complete evidence graph for facts.

4.4 Module 4: Multi-Factor Path Scorer and Contradiction Engine

File: `src/path_scorer.py` **Status:** In scope - Week 4

- Score paths using relevance, evidence quality, anchor coverage, diversity, length, and contradiction signals.
- Detect claim-level conflicts in predicates, numerical values, and temporal qualifiers.
- Retain top-K paths together with contradiction flags instead of discarding opposing evidence.

4.5 Module 5: Provenance Traceback Engine

File: `src/provenance_tracker.py` **Status:** In scope - Week 5

- Resolve selected path claim IDs against Module 2's provenance records.
- Recover exact document, chunk, sentence, and character spans.
- Return supporting and contradicting source text together.

4.6 Module 6: Deduplication and Context Pruner

File: `src/context_pruner.py` **Status:** In scope - Week 6

- Deduplicate chunks referenced by overlapping paths.
- Keep only sentences that support or contradict selected claims.
- Attach inline citations and assemble the compact final evidence payload.

5. Mathematical Formulations Reference

Measure	Definition
Multi-factor path score	Score(P,Q) = w1 Rel + w2 Qual + w3 Cov + w4 Div - w5 Len - w6 Contra
Claim coverage	Coverage(P) = |Claims(P) intersect QueryAnchors(Q)| / |QueryAnchors(Q)|
Diversity penalty	max over j<k of |Nodes(Pk) intersect Nodes(Pj)| / |Nodes(Pk)|

6. 8-Week Engineering Implementation Roadmap

Week	Deliverable	Status
1	Two-tier query parser, input sanitation, entity and relation anchors	Complete
2	Provenance-aware MultiDiGraph builder, chunking, canonical resolution, validated SVO claims, connected backbone, serialization, evaluation, plotting	Complete
3	Query-aware bounded path traverser and seed mapping	Next
4	Multi-factor scoring and contradiction engine	Planned
5	Path-to-source provenance traceback	Planned
6	Deduplication, sentence pruning, and citation formatting	Planned
7	Pipeline integration, final LLM orchestration, and interactive CLI	Planned
8	Benchmarking: faithfulness, citation correctness, contradiction recall, and token reduction	Planned

Roadmap interpretation

The Module 2 `main.py` runner is a development convenience that combines build, export, evaluation, and plotting. It does not satisfy Week 7's final query-to-LLM orchestration requirement.

7. Success and Acceptance Criteria

Criterion	Target	Current evidence
Module 2 traceability	100% of accepted claims map to source spans	Provenance round-trip check passes
Module 2 connectivity	One traversable component with non-factual bridges clearly marked	1 component; sparse backbone passes
Module 2 retrieval readiness	Document-derived entity/relation queries recover exact supporting claims	4 of 4 fixture retrieval cases pass
Contradiction recall	At least 80% precision on test corpora	Pending Module 4
Token reduction	30% to 50% versus naive chunk retrieval	Pending Module 6
Answer faithfulness	Improve citation accuracy over Vector RAG and GraphRAG baselines	Pending integrated benchmark

MODULE 2 ACCEPTANCE DECISION

Accepted for progression to Module 3. The graph builder meets the current document-level requirements for connected structure, provenance retention, canonical alias handling, malformed-triple rejection, factual claim storage, serialization, retrieval checks, and traversal visualization.